POLYMATH-ATLAS
OSA-4.0

UNIFIED ARCHITECTURAL CODEX

От хаоса лексем к формальному семантическому ядру.
Полная хронология синтеза + формальное ядро + законы + Torture Set.

REFERENCE • 08 AUGUST 2026

РЕМАРКА: Meta-Schema НЕ заморожена. Не генерировать до замыкания Kernel → Lawbook → Model Check.

0. EXECUTIVE STATUS

Контур	Статус	Комментарий
OSA-3.x	LEGACY	Концептуальный слой; его дефекты инициировали Formal Core.
Kernel v0.1–0.3	SUPERSEDED	Type Algebra → Effects → Predicate Liveness.
T01–T15	AUDIT ≠ PROOF	Контрпримеры выявили дыры. 15/15 ≠ soundness.
Kernel v0.3.1 / v0.4	CURRENT BASE	WP, Join Semilattice, Presence≠Truth, Identity, Authorization.
Meta-Schema	DEFERRED	Сериализация математики, а не место её появления.

РЕМАРКА: Нельзя объявлять Kernel sound только потому, что заранее придуманные тесты дали ожидаемые verdicts.

1. ЧТО ТАКОЕ POLYMATH-ATLAS

Попытка построить типизированное операционное пространство для познания, анализа, инженерии, проверки и рефлексии. Центральный переход: не «слово = смысл», а «лексема = кандидат на операционный примитив» → тип, вход, выход, контракт, инвариант, механизм, эффекты, неопределённость, допустимая композиция.

РЕМАРКА: Сильная сторона — переход от словаря терминов к системе допустимых преобразований между состояниями, моделями, решениями и действиями.

```
ENTITY / OPERATION / RESULT / PROPERTY / METHOD / ARTIFACT / RELATION / PROCESS
                          ↓
                  TYPE + CONTRACT
                          ↓
                STATE TRANSITION
                          ↓
             COMPOSITION / VALIDATION
                          ↓
                EXECUTION / REPAIR
```

2. ПРОЛОГ: ИСХОДНОЕ СОСТОЯНИЕ И КРИЗИС

Проект начался с попытки формализовать универсальный цикл познания через каталог из 418 русских глаголов. Первичный аудит выявил фундаментальный дефект: смешение в одном списке операций, результатов, свойств, артефактов и дисциплин. Линейная цепочка «Целое → Выделить → … → Новое Целое» — лишь один из workflow-паттернов, а не универсальный закон.

РЕМАРКА: Отвергнуто: операционная алгебра напрямую из естественного языка. Словарь без строгой селекции — свалка. Лингвистический детерминизм порождает этноцентрическую онтологию.

3. АРХИТЕКТУРНЫЙ СДВИГ — 5 ОРТОГОНАЛЬНЫХ ОСЕЙ

Вместо одной иерархии L0–L4 введены пять независимых осей:

- Ось A (Категория): Object, Data, Model, Decision, Action, Meta
- Ось B (Роль): Input, Output, Resource, Target, Constraint
- Ось C (Абстракция): L0 Reality → L1 Data → L2 Model → L3 Decision → L4 Action
- Ось D (Тип): формальная решётка IsSubtype(A, B)
- Ось E (Фаза): Created, Available, Consumed, Destroyed

4. OSA-3.x → ПЕРЕХОД К FORMAL CORE

Локальные строковые патчи не закрывали системные дефекты: отсутствие доказуемой типовой системы, логики предикатов, модели времени/ресурсов, параллелизма, векторных эффектов, агрегации неопределённости, Repair Planner, теории идентичности.

Дефект	Почему критичен
Типовая система	IsSubtype не доказан → бесконечный patching
Preconditions	Строки ≠ логический вывод
Destroys	Неясно: объект / факт / свойство / состояние
Uncertainty	«inherit» — placeholder, не контракт
Invariant Engine	Пересечение required/destroyed — sanity check, не теория
Testing	15 негативных кейсов ≠ proof of soundness
Formal Verification	Lean/Coq проверяет код vs spec, не истинность spec
Repair	BFS без контрактов — патчер, не planner
Identity	RESTORE/RECREATE требуют различия identity и lineage

РЕМАРКА: Именно эти дефекты определяют, почему OSA-4.0 начинается с семантики, а не с JSON Schema.

5. ЭВОЛЮЦИЯ SEMANTIC KERNEL

5.1 Kernel v0.1 — Formal Core

Четыре измерения сущности + TypeLattice + Predicate + EvalResult (Proven True / False / Unknown) + State + Operator.

```
Predicate = FactExists | Not | And | Or | Implies | Gte | Eq | ForAll | Exists
EvalResult = Proven True | Proven False | Unknown
State = { facts, objects, resources, permissions, provenance, uncertainty, temporalValidity }
```

РЕМАРКА: Главный прорыв: предусловие становится вычисляемым предикатом; недостаток данных → Unknown, а не ложный Reject.

5.2 Kernel v0.2 — Execution + Effects

Fact с provenance/epistemic/validity/owner; OperatorEffect (produces/consumes/preserves/invalidates/modifies/requires); TriBool; goal-relative liveness; branch-aware intersection; constrained Repair Planner.

```
Verdict:
  any FALSE          → REJECT
  all TRUE           → PASS
  valid repair (no Unknown) → REPAIR
  repair has Unknown → UNDECIDABLE
  otherwise          → REJECT
```

РЕМАРКА: Set subtraction достаточен для FactID, но недостаточен для логических предикатов → WP в v0.3.1.

5.3 Kernel v0.3 — Predicate & Lattice

Predicate-level Liveness + эпистемическая решётка (TRUE_KNOWN, FALSE_KNOWN, UNKNOWN, ABSENT, CONFLICT) + lexicographic Repair.

```
TRUE_KNOWN ⊔ FALSE_KNOWN = CONFLICT
TRUE_KNOWN ⊔ UNKNOWN     = UNKNOWN
TRUE_KNOWN ⊔ ABSENT      = UNKNOWN
```

5.4 Kernel v0.3.1 / v0.4 — Algebraic Closure (CURRENT)

Пять независимых пространств: Type, State, Effect, Epistemic, Predicate. Presence отделена от PredicateTruth. WP-трансформер вместо set subtraction. Полная Join-semilattice. Recreation ≠ Resurrection. Effect Authorization. Predicate Invalidation.

```
ResourcePresence = Present | Absent
PredicateTruth   = TRUE_KNOWN | FALSE_KNOWN | UNKNOWN | CONFLICT

WP(Op, Q):
  S □ WP(Op,Q)  iff  Exec(Op,S) ↓ S'  ∧  S' □ Q
LiveBefore(Op,Q) = Pre(Op) ∧ WP(Op,Q)
```

РЕМАРКА: ABSENT намеренно вынесен из Join-semilattice. Invalidates(P) ≠ proves ¬P.

6. EPISTEMIC JOIN-SEMILATTICE

Строгое замыкание таблицы слияния ветвей. CONFLICT — топовый элемент. Доказаны идемпотентность, коммутативность, ассоциативность.

⊔	TRUE	FALSE	UNKNOWN	ABSENT	CONFLICT
TRUE	TRUE	CONFLICT	UNKNOWN	UNKNOWN	CONFLICT
FALSE	CONFLICT	FALSE	UNKNOWN	UNKNOWN	CONFLICT
UNKNOWN	UNKNOWN	UNKNOWN	UNKNOWN	UNKNOWN	CONFLICT
ABSENT	UNKNOWN	UNKNOWN	UNKNOWN	ABSENT	CONFLICT
CONFLICT	CONFLICT	CONFLICT	CONFLICT	CONFLICT	CONFLICT

РЕМАРКА: ABSENT (объекта нет в ветви) ≠ FALSE (доказано, что предикат ложен).

7. АЛГЕБРА ИДЕНТИЧНОСТИ

Запрещён неоднозначный RESTORE(X). Строгое разделение:

```
Resurrection (ЗАПРЕЩЁН):
  Destroyed(X) → Available(X)   same identity   → REJECT

Reconstruction / Recreation (РАЗРЕШЁН):
  Destroyed(X_old) → X_new : Available
  ID(X_new) ≠ ID(X_old) + Provenance          → potentially legal
```

8. LEXICOGRAPHIC REPAIR OPTIMIZER

```
V = [SafetyViolations, ResidualProofObligations, EffectPolicy, Uncertainty, Cost]
V1 = [1, 0, 0, 0, 1]   // Unsafe, Cheap
V2 = [0, 0, 0, 0, 3]   // Safe, Expensive
Выбор: V2  (Safety доминирует)
```

9. LAWBOOK — 17 АКСИОМ (готовы к Lean 4 / Coq)

ID	Закон	Формула
LAW-001	Type Reflexivity	∀A ∈ T, A ⊑ A
LAW-002	Type Transitivity	A ⊑ B ∧ B ⊑ C ⇒ A ⊑ C
LAW-003	State Conservation	Op не меняет X ⇒ State_after(X)=State_before(X)
LAW-004	No Resurrection	Destroyed → Available с тем же ID запрещено
LAW-005	Consume Safety	consume допустим ⇔ Available
LAW-006	Predicate Liveness Soundness	KILL={P} допустим ⇔ P ∉ LIVE_OUT
LAW-007	Semilattice Soundness	Eval_merge(P) = ⊔ Eval_branches(P)
LAW-008	Effect Monoid Associativity	E(G) инвариантен к группировке
LAW-009	Epistemic Non-Overclaim	U_out < U_in требует явного Evidence
LAW-010	Composition Soundness	A;B допустима ⇔ существует валидный S'
LAW-011	Effect Authorization	RequiresAuth ⇒ Auth ∈ LIVE_IN
LAW-012	WP Transformer Soundness	{WP(Op,Q)} Op {Q}
LAW-013	Join Associativity	(x⊔y)⊔z = x⊔(y⊔z)
LAW-014	Join Commutativity	x⊔y = y⊔x
LAW-015	Join Idempotence	x⊔x = x
LAW-016	Recreation Identity	RECREATE ⇒ ID(out) ≠ ID(in)
LAW-017	Predicate Invalidation	Invalidates(P) ⇒ P ∈ KILL

10. SEMANTIC TORTURE SET — КЛЮЧЕВЫЕ РЕЗУЛЬТАТЫ

T03 DELETE(X) → ANALYZE(X) → REJECT (Liveness)

T05 MEASURE → MODEL (Sufficiency=UNKNOWN) → UNDECIDABLE

T09 Branch TRUE ⊔ FALSE → CONFLICT → REJECT

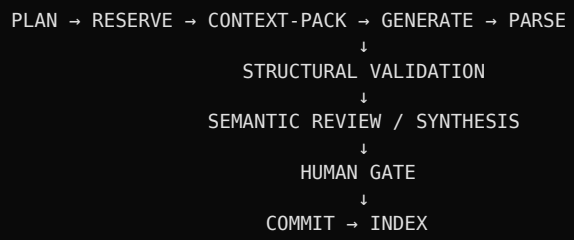
T11 INVALIDATE(Calibrated) → USE → REJECT

T04 Resurrection same-ID → REJECT; Recreation new-ID → допустимо

T15 Cheap unsafe vs Safe expensive → выбирается Safe (lexicographic)

РЕМАРКА: 15/15 — аудит, а не доказательство soundness. Следующий шаг: minimal basis set по каждому закону (positive / negative / boundary / branch / repair).

11. ENGINEERING PIPELINE v5



LLM предлагает содержание, но не управляет графом. ID, ссылки, топология и принятие — машинный слой.

РЕМАРКА: При 301+ узлах полный граф нельзя постоянно подавать LLM. Нужен локальный context pack + компактный глобальный индекс.

12. ОТКРЫТЫЕ ПРОБЛЕМЫ И ГРАНИЦЫ

Интерпретация — контекст бесконечен, смысл не типизируется. Оставлена как сквозной комбинатор.

Модель Агента — компетенция и юрисдикция — социальные конструкты.

Валидация Цели — формальная система не оценивает этичность/истинность самой цели.

13. ФИНАЛЬНАЯ ТРАЕКТОРИЯ

```
OSA-3.x
   ↓ AUDIT / DEFECT CATALOG
FORMAL CORE v0.1
   ↓
EXECUTION KERNEL v0.2
   ↓
PREDICATE & LATTICE v0.3
   ↓ TORTURE SET T01–T15
COUNTEREXAMPLES → SEMANTIC GAPS
   ↓
KERNEL v0.3.1 / v0.4
   ├── PredicateSpace + WP
   ├── Presence ≠ Truth
   ├── complete Join Semilattice
   ├── Recreation ≠ Resurrection
   ├── Effect Authorization
   ├── Predicate Invalidation
   └── Repair Soundness
   ↓
LAWBOOK (17 axioms)
   ↓
MINIMAL BASIS TORTURE SET
   ↓
MODEL CHECK
   ↓
KERNEL v0.4  (closed)
   ↓ ONLY THEN
Atlas S-1.0 Meta-Schema
```

РЕМАРКА: Meta-Schema должна быть сериализацией уже определённой математики, а не местом, где математика впервые появляется.

Polymath-Atlas — не универсальный алгоритм мышления и не готовое ядро AGI. Это строгая типизированная операционная семантика, способная строить, проверять и аудировать графы преобразований, честно фиксируя собственные эпистемические границы.

Compiled by Polymath-Architect • OSA-4.0 Unified Codex • 08 Aug 2026